AI Agent Security Standards

The controls we enforce across every AI agent we build and operate.

Document	AI Agent Security Standards — Enforcement Reference
Audience	Engineering, security reviewers, partners, auditors
Issued	May 2026
Frameworks	OWASP AI Agent Security Cheat Sheet (ASI01–ASI10); AI Agent Security Workshop — 10 Golden Rules; OWASP Top 10 for LLM Applications
Status	Enforced — every agent project is reviewed against this checklist.

Purpose. This document describes the security controls applied to every AI agent project. Each control area maps to an OWASP AI category or workshop rule, and each item is verified against the live codebase during review — not against documentation alone.

How we verify. A read-only security review is performed against each project. Every control is rated **COVERED**, **PARTIAL**, or **MISSING** with cited evidence (file and function). Gaps produce a prioritised remediation queue.

Contents

1. Review Methodology

Each agent project is assessed in three phases. The review is strictly read-only — source code is never modified during assessment.

Phase	Activity
1. Architecture	Read project documentation, entry points, module layout, dependencies, container and CI/CD configuration. Build a mental model of external systems, credentials, data flow, and whether the pipeline is stateless or maintains memory.
2. Evidence	For each control, search the codebase for the actual implementation. Cite the file path and function name as evidence. A control is never rated COVERED based on documentation alone.
3. Reporting	Produce *SECURITY_CHECKLIST.md* with every control rated and evidenced, plus *SECURITY_FIXES_TODO.md* prioritising gaps as High / Medium / Low risk with specific remediation steps.

2. Rating Scale

Rating	Meaning
COVERED	The control is demonstrably implemented in code. Evidence cites file and function.
PARTIAL	The control is partially addressed; the specific gap is noted with remediation steps.
MISSING	No implementation found in the codebase. Treated as a gap requiring action.
N/A	Not applicable to this architecture (e.g., no UI, no persistent memory).

3. ASI01 — Prompt Injection

We treat every piece of content that did not originate from a trusted developer or system component as potentially adversarial. Untrusted text must never be allowed to override instructions, exfiltrate context, or redirect tool calls. The controls below define how untrusted input is bounded, sanitised, and confined to the appropriate message role.

ID	Control
1.1	All external data is treated as untrusted by default.
1.2	Clear delimiters separate instructions from data inside prompts.
1.3	A content filter (regex or classifier) screens known injection patterns.
1.4	A separate LLM pre-pass validates untrusted content where warranted.
1.5	Indirect injection vectors (Jira, Orchestrator, DB responses) are sanitised before reaching the LLM.
1.6	Untrusted input is passed in the *user* role — never injected into the system prompt.

4. ASI02 — Tool Misuse & Exploitation

Agents must only reach the systems they need, with the minimum permissions required. LLM output is never passed unmodified into a tool call. Every external query is built from validated arguments, and every integration is rate-limited to prevent abuse or runaway loops.

ID	Control
2.1	Minimum tool set — the agent cannot reach systems it does not need.
2.2	Read-only scopes for every external integration unless writes are explicitly required.
2.3	Tool arguments are validated and escaped (JQL, SQL, OData) before each call.
2.4	LLM output is never used raw as a tool argument.
2.5	Rate limits and hard caps on tool calls per session.

5. ASI03 — Identity & Privilege Abuse

Every agent runs under its own scoped service identity, distinct from any human account. Credentials are short-lived where the platform allows, never shared across environments, and every action is attributable to a specific run.

ID	Control
3.1	The agent has its own scoped service identity — no shared human credentials.
3.2	Short-lived, rotated credentials are used where the platform supports them.
3.3	Audit logs attribute every action to a specific run and identity.

3.4	No credential sharing across agents or environments.

6. ASI04 — Supply Chain

Software dependencies, container base images, and third-party model providers are all part of the trust surface. We pin versions, scan images, and document every flow of data to providers outside our cloud tenant.

ID	Control
4.1	Dependencies are pinned to exact or compatible-release versions.
4.2	Container images are scanned in CI (Trivy, Grype, or Snyk).
4.3	Base images use SHA digests or verified official images.
4.4	Third-party LLM data egress is documented and approved.
4.5	No unapproved MCP servers or third-party plugins.

7. ASI05 — Unexpected Code Execution

LLM output is text — never executable. No agent in our environment passes model-generated content to eval(), exec(), a subprocess, or a browser without strict, separately-reviewed sandboxing.

ID	Control
5.1	No eval / exec / subprocess execution of LLM-generated content.
5.2	Agent-generated code is never auto-executed on the host.

8. ASI06 — Memory & Context Poisoning

Where an agent has no business need for persistent memory, it has none. Where memory is necessary, it is scoped, encrypted, and prevented from forming self-reinforcing loops in which an agent's outputs poison its future inputs.

ID	Control
6.1	No persistent memory between runs — or memory is scoped and encrypted.
6.2	Each run is fully isolated (unique run ID, temp workspace).
6.3	No self-reinforcing loops — agent outputs are not written back as inputs.

9. ASI08 — Cascading Failures

A single failure must never propagate into a runaway state. Every LLM call has a retry limit, a timeout, and a token budget; any partial failure yields a safe default rather than a crash or corrupted output.

ID	Control
8.1	Retry limits on LLM and external API calls.
8.2	Token budget enforcement with a hard stop.
8.3	LLM call timeouts (thread-based or async).
8.4	Fail-safe defaults — partial failure yields UNKNOWN, not crash.

10. Data Protection

Personal data must not be exposed to the LLM or written to output unnecessarily. Masking runs on the way in, output is scanned on the way out, upload sizes are bounded, and retention is governed by policy.

ID	Control
DP1	PII masking runs before every LLM call.
DP2	Entity list covers the relevant jurisdiction (name, email, phone, national ID, etc.).

DP3	LLM output is scanned for PII before being written to disk.
DP4	Upload and input size limits are enforced.
DP5	Retention and deletion policies exist for outputs and audit logs.
DP6	Privacy notice is presented in the UI where applicable.

11. Infrastructure & Container

Containers run as non-root, secrets stay out of images, and any user interface either requires authentication or is explicitly scoped as internal-only. XSRF/CSRF protections are enabled on web-facing components.

ID	Control
IC1	Container runs as a non-root user.
IC2	No secrets baked into the container image.
IC3	XSRF / CSRF protection enabled.
IC4	UI requires authentication, or is explicitly internal-only.

12. Monitoring & Audit

Every run is logged immutably with the fields needed to reconstruct what happened. Logs are stored in a separate trust boundary from the agent, and alerts fire on token spikes, repeated failures, and other security-relevant events.

ID	Control
M1	Immutable audit log is written before output becomes accessible.
M2	Audit log captures run ID, input source, PII counts, token usage, and status.
M3	Logs are stored in a separate trust boundary from the agent.
M4	Alerts fire on security-relevant events (failures, token spikes, unknown fields).

13. Incident Response

Every project has a documented procedure for taking the agent offline quickly, revoking credentials at every external integration, and escalating through a known contact chain.

ID	Control
IR1	Kill-switch procedure is documented (agent offline in under 60 seconds).
IR2	Incident response runbook: Detect → Contain → Eradicate → Recover.
IR3	Credential revocation steps for every external integration.
IR4	Contacts and escalation path are documented.

14. Adversarial Testing

Security controls are tested, not merely declared. Injection fixtures and PII leakage tests run in CI; tests assert that injected instructions are *not* reflected in agent output.

ID	Control
AT1	Injection test fixtures exist.
AT2	Tests assert that injected instructions are not reflected in output.
AT3	PII leakage tests exist.
AT4	Adversarial tests run in CI — not excluded.

15. Review Deliverables

Each security review produces two artefacts, written to the project root and committed to version control:

Artefact	Contents
SECURITY_CHECKLIST.md	Every control above, rated COVERED / PARTIAL / MISSING / N/A, with the cited evidence (file path and function) or a documented reason for the rating. Includes a summary table and one-line overall posture statement.
SECURITY_FIXES_TODO.md	Generated when any gaps exist. Prioritised as *Fix Immediately (High)*, *Fix Soon (Medium)*, or *Fix When Infrastructure is Ready (Low)*. Each entry names the exact file, the snippet to add, and where to place it.

Questions, or requesting a review? Contact the Security team. Reviews are scheduled before any agent project moves to production, and re-run whenever the agent's trust surface materially changes.